# Lab 05 — Antwoorden met toelichting

*Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.*

---

### Vraag 1 — 950 MB geweigerd door de beveiliging

**Antwoord.** (1) **De broncode zit in de image**: wie de image pullt, kan de applicatie lezen — en vaak ook lokale configuratiebestanden. (2) **Het aanvalsoppervlak**: JDK, Maven, `git`, `curl`, een volledige shell, honderden Debian-pakketten — stuk voor stuk gereedschap voor een aanvaller die code kan uitvoeren, en samen goed voor een scanrapport met 300 CVE's dat niemand doorwerkt. (3) **De repository** `~/.m2` bevat de gedownloade artefacten en, vaak genoeg, een `settings.xml` met de logingegevens van de private Maven-repository. Zonder multi-stage is (2) het moeilijkst weg te werken: bronnen en `.m2` kun je met `rm` verwijderen (slecht — de lagen houden ze bij, lab 02), maar de JDK krijg je niet uit een image die van een JDK-image vertrekt.

**Waarom.** De uiteindelijke image is de laatste `FROM` plus alles wat je erbovenop legt; de basisimage "aftrekken" kan niet. Alleen een tweede `FROM` op een minimale basis, gecombineerd met `COPY --from`, verandert de basis.

**Nuance.** Grootte heeft wel degelijk een operationele kost (pulltijd bij een incident, opslag in de registry), maar tegenover een veiligheidsverantwoordelijke is het je zwakste argument — wat er in de image zit, weegt zwaarder dan wat ze weegt.

**Voorbeeld.**

```
podman run --rm --entrypoint sh api-mono:1.0 -c 'ls /app; javac -version; ls ~/.m2 2>/dev/null'
```

---

### Vraag 2 — Drie `FROM`s

**Antwoord.** De **laatste** `FROM` levert de uiteindelijke image; de twee andere zijn tijdelijke omgevingen die aan het einde van de build vernietigd worden (alleen hun cache blijft over). Verwijst geen enkele `COPY --from` (en geen latere `FROM`) naar de tweede stage, dan **bouwt Buildah ze helemaal niet**: ze wordt overgeslagen. Dat zie je in de uitvoer: de stages zijn genummerd `[1/3]`, `[2/3]`, `[3/3]`, en het nummer van de ongebruikte stage verschijnt nergens.

**Waarom.** De engine leidt eerst uit de `--from`-verwijzingen de afhankelijkheidsgraaf tussen de stages af, en bouwt dan alleen wat naar het doel leidt (de laatste stage, of de stage van `--target`).

**Nuance.** BuildKit doet hetzelfde, en bouwt onafhankelijke stages bovendien parallel. Een "ongebruikte" stage heeft ook een legitiem doel: een teststage (`RUN mvn test`) die de CI alleen bouwt met `--target test`.

**Voorbeeld.**

```
podman build -f Dockerfile.unused -t u . 2>&1 | grep STEP     # alleen [2/3] en [3/3], nooit [1/3]
```

---

### Vraag 3 — 420 MB en de bronnen

**Antwoord.** `COPY --from=build /app /app` kopieert **de volledige werkmap** van de stage `build`: `Api.java` / `src`, `pom.xml`, `target/` met alle klassen, en de JAR. De fix: kopieer alleen het artefact.

```
COPY --from=build /app/target/api.jar /app/api.jar
```

(en pas de `ENTRYPOINT` aan naar `/app/api.jar`.)

**Waarom.** Multi-stage filtert zelf niets: de uiteindelijke image krijgt precies wat `COPY --from` vraagt. Vraag je een map, dan krijg je alles wat erin zit.

**Nuance.** Zit er in de gecorrigeerde image nog een `.m2`? Nee — `.m2` staat in `/root` van de build-stage, niet in `/app`. Maar dat is geluk, geen garantie. De regel: kopieer **bestanden bij naam**.

**Voorbeeld.**

```
podman run --rm --entrypoint ls api-multi:1.0 /src     # No such file or directory: goed teken
```

---

### Vraag 4 — `ng serve` in productie

**Antwoord.** Vier redenen: (1) `ng serve` is een **ontwikkel**server — niet geoptimaliseerd, geen compressie, geen HTTP-cache, en de eigen documentatie zegt uitdrukkelijk dat hij niet voor productie bedoeld is; (2) de image bevat **Node, de bronnen en `node_modules`** (vaak meer dan 1 GB): een groot aanvalsoppervlak én een codelek; (3) de build gebeurt niet **één keer** maar bij elke start, in watch-modus, met *source maps* aan; (4) applicatie en tooling zijn niet gescheiden — een kwetsbare ontwikkelafhankelijkheid draait zo mee in productie. In de plaats: draai `ng build --configuration production` in een Node-stage, en kopieer daarna met `COPY --from` de map `dist/…/browser` naar een image `nginx:alpine`, met een nginx-configuratie die `index.html` teruggeeft voor de Angular-routes.

**Waarom.** Een gecompileerde Angular-frontend is statisch. Om die te serveren volstaat een bestandsserver; al de rest is buildwerk, en buildwerk hoort in de weggegooide stage.

**Nuance.** Er is één geval waarin Node in productie blijft: server-side rendering (Angular SSR / Universal). Dat is dan een **andere** applicatie met een eigen Dockerfile — en nog altijd geen `ng serve`.

**Voorbeeld.**

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'   # 64 MB
tegenover 167 MB (zonder node_modules!)
```

---

### Vraag 5 — `UnsatisfiedLinkError` na Alpine

**Antwoord.** De PDF-generatie leunt vrijwel zeker op een **native bibliotheek** (`.so`) die voor `glibc` gecompileerd is — lettertypen, rendering, compressie. Alpine levert `musl`: de loader weigert de bibliotheek, en Java gooit een `UnsatisfiedLinkError`. Het commando dat het verschil had blootgelegd: `ldd --version` in elke image (`musl libc` tegenover `GLIBC`). De migratie had getest moeten worden met de **echte workloads** (niet alleen `/actuator/health`), op een acceptatieomgeving, met een terugvalplan — en component per component beslist.

**Waarom.** Een native bibliotheek is gebonden aan één specifieke `libc`; het is geen portable bytecode. Dat het twee weken duurde, is geen raadsel: de PDF-taak draait misschien alleen aan

het einde van de maand.

**Nuance.** Er zijn alternatieven: de image `eclipse-temurin:21-jre-ubi9-minimal` (Red Hat, `glibc`, ~100 MB), of `gcompat` op Alpine (fragiel). En het is geen zuiver Java-probleem: Python en Node met native modules trappen in exact dezelfde val.

**Voorbeeld.**

```
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre-alpine -c 'ldd --
version 2>&1 | head -1'   # musl libc
podman run --rm --entrypoint sh docker.io/library/eclipse-temurin:21-jre -c 'ldd --version |
head -1'              # GLIBC 2.xx
```

---

### Vraag 6 — Multi-stage en geheimen

**Antwoord.** Een `rm` maakt een laag aan die het bestand verbergt, maar de laag die het schreef, blijft in de image zitten (lab 02). Een weggegooide stage daarentegen maakt **geen** deel uit van de uiteindelijke image: geen enkele van haar lagen zit erin. Een geheim dat alleen in een weggegooide stage geschreven werd, bestaat dus nergens in het gepubliceerde artefact. Multi-stage beschermt niet meer zodra het geheim naar de laatste stage **gekopieerd** wordt (`COPY --from=build /app` met het geheim erin), zodra het artefact het zelf opgeslorpt heeft (een `application.yml` met wachtwoord die in de JAR zit), of zodra het geheim via een `ARG` van de laatste stage passeert (zichtbaar in `history`).

**Waarom.** Uiteindelijke image = de lagen van de laatste `FROM` + de lagen die zijn instructies aanmaken. Een eerdere stage draagt alleen bij wat een `COPY --from` eruit haalt.

**Nuance.** De moderne aanpak is `RUN --mount=type=secret`: het geheim is één instructie lang beschikbaar, in eender welke stage, en wordt nooit een laag. Multi-stage blijft de structurele garantie; de *secret mount* is de garantie per instructie.

**Voorbeeld.**

```
podman build --secret id=pw,src=pw.txt -f Dockerfile.secret -t sec .
podman run --rm sec ls /run/secrets      # No such file or directory
```

---

### Vraag 7 — JAR in één blok tegenover lagen

**Antwoord.** (a) Eén laag van 50 MB die bij elke build verandert: de `push` en elke `pull` verplaatsen **50 MB**. (b) Vier lagen — afhankelijkheden (~45 MB, ongewijzigd), loader (~1 MB, ongewijzigd), snapshots (0), applicatie (~5 MB) — dus een uitrol verplaatst **~5 MB**. Een factor tien. Toch blijft (a) aanvaardbaar: 50 MB gaat op een datacenternetwerk in ongeveer een seconde over de lijn, de JRE (180 MB) wordt hoe dan ook gedeeld, en (b) voegt een extra stage toe, een andere `ENTRYPOINT` (`org.springframework.boot.loader…` of `java -jar` op de map), en complexiteit die iemand moet uitleggen.

**Waarom.** De overdracht gebeurt laag per laag, differentieel. Wat telt, is de grootte van de laag die verandert, niet die van de image.

**Nuance.** (b) begint te lonen zodra je vaak uitrolt naar veel nodes, of over een traag netwerk (edge, afgelegen locaties). Het principe werkt ook zonder Spring: `lib/` (stabiel) en `classes/` (vluchtig) scheiden volstaat al.

**Voorbeeld.**

```
podman history api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' | head -6   # één laag van 50 MB, of vier lagen
```

---

### Vraag 8 — 90 seconden die 7 minuten werden

**Antwoord.** De nieuwe agent start met een **lege cache**: de buildcache (lagen) leeft op de machine die bouwt, dus een verse agent — of een wegwerpagent die bij elke pipeline opnieuw wordt aangemaakt — begint van nul. De 5 minuten van `dependency:go-offline` worden dus opnieuw betaald. Twee mechanismen: (1) een **cache mount** (`RUN --mount=type=cache,target=/root/.m2`), die de Maven-repository tussen builds op de agent bewaart, ook als `pom.xml` verandert; (2) een **externe cache** — `--cache-from` / `--cache-to` richting registry — waarmee een verse agent de lagen van een vorige build kan ophalen. Met rootless Podman zit de cache (lagen en *cache mounts*) in `~/.local/share/containers/storage` van de gebruiker die bouwt: een agent die elke job onder een andere gebruiker of `home` draait, of die zijn `home` weggooit, heeft nooit een cache.

**Waarom.** "Er is niets veranderd" klopt voor de bronnen, maar niet voor de cache: de cache is lokale toestand van de machine, geen eigenschap van de Dockerfile.

**Nuance.** Wegwerpagents zijn een bewuste keuze (isolatie, reproduceerbaarheid); de oplossing is de cache **expliciet en extern** maken, niet agents langer in leven houden. En een `--no-cache` die "voor de zekerheid" in de pipeline staat, veroorzaakt exact dit symptoom — permanent.

**Voorbeeld.**

```
podman build --cache-to registry.intern/mijnapp/api-cache --cache-from
registry.intern/mijnapp/api-cache -t api:1.5 .
podman info --format '{{.Store.GraphRoot}}'    # waar de cache van deze gebruiker leeft
```

---

### Vraag 9 — Wat je verliest met distroless

**Antwoord.** (1) **`podman exec -it … sh`**: geen shell betekent geen interactieve verkenning, geen `cat` op een configuratiebestand, geen `curl localhost:8080/actuator`. Teams vangen dat op met blootgestelde observeerbaarheidsendpoints (`/actuator/health`, `/info`, `/env`), volledige gestructureerde logs op `stdout`, en `podman cp` om een bestand uit de container te halen. (2) **Diagnosetools** (`jcmd`, `jstack`, `ps`, `netstat`): niets om een *thread dump* mee te nemen of de sockets te bekijken. Teams vangen dat op met een debug-*sidecar*-container die de namespaces deelt (`podman run --pid=container:api --network=container:api debug-image`), of met JMX/Actuator-tooling (`/actuator/threaddump`) op het interne netwerk.

**Waarom.** Elk gereedschap waarmee een beheerder een container binnenraakt, dient een aanvaller net zo goed. Distroless schrapt beide tegelijk; de observeerbaarheid moet dus **buiten** de image komen te liggen.

**Nuance.** Distroless images bestaan in een `:debug`-variant met een busybox-shell — handig in acceptatie, verboden in productie. En Kubernetes heeft `kubectl debug` met kortlevende containers, precies voor deze behoefte.

**Voorbeeld.**

```
podman exec d sh -c ls          # executable file `sh` not found
curl -s localhost:18082/actuator/health     # de observeerbaarheid verloopt via HTTP
```

### Vraag 10 — De cache mount, `VOLUME`, en `# syntax=`

**Antwoord.** De gegevens staan in een **cachemap die de build-engine beheert** (BuildKit of Buildah), op de machine die bouwt — bij rootless Podman in je gebruikersopslag. Die map wordt **alleen tijdens de instructie** in de buildcontainer gemount en daarna weer losgekoppeld: er wordt niets in een laag geschreven, dus er belandt niets in de image. Een `VOLUME` is het omgekeerde: een declaratie die in de image zit en pas bij de **uitvoering** een volume voor de container aanmaakt; tijdens de build doet ze niets. Zonder `# syntax=docker/dockerfile:1`: recente Docker-versies (BuildKit standaard) draaien `--mount` gewoon met de huidige stabiele syntax — de regel diende alleen om een nieuwere frontend-versie af te dwingen. Podman **negeert** de regel volledig (Buildah heeft geen frontend), en `--mount` werkt er van nature.

**Waarom.** De cache mount is een mechanisme van de build-engine; `VOLUME` een mechanisme van de runtime. Het woord "mount" is het enige wat ze delen.

**Nuance.** De cache mount wordt niet gedeeld tussen machines of gebruikers, en de inhoud wordt nooit ongeldig verklaard: een corrupte Maven-repository blijft er gewoon in staan. `podman system prune --build-cache`… bestaat nog niet: je verwijdert de opslag, of je wisselt van cache met `id=`.

**Voorbeeld.**

```
podman build --no-cache -f Dockerfile.cache -t c . 2>&1 | grep dep-    # de markers stapelen zich op van build tot build
podman run --rm c ls /root/.m2                                         # afwezig in de image
```

---

### Vraag 11 — "Multi-stage dient tot niets voor Angular"

**Antwoord.** Zonder multi-stage is de uiteindelijke image de image waarin `ng build` gedraaid heeft: `node:22-alpine` (~170 MB), **plus** `node_modules` (500 MB tot 1 GB), **plus** de TypeScript-bronnen, **plus** `dist/` — en dan moet er nog een server bij om `dist/` te serveren. Met multi-stage: `nginx:alpine` (64 MB) plus enkele MB statische bestanden. Dat scheelt een factor 10 tot 20, en de inhoud verandert van aard: geen Node meer, geen bronnen, geen buildafhankelijkheden.

**Waarom.** Dat het resultaat statisch is, is net het argument **vóór** multi-stage: als de uitvoering niets nodig heeft van wat de build gebruikte, is er geen reden om er iets van te bewaren.

**Nuance.** Zonder containerbuild kan een team ook `ng build` in de CI draaien en `dist/` in één stap naar een nginx-image kopiëren (`COPY dist/ /usr/share/nginx/html`). Dat is een "multi-stage" waarvan de eerste stage de CI zelf is — geldig, maar de build is dan niet meer reproduceerbaar vanuit de Dockerfile alleen.

**Voorbeeld.**

```
podman images --format '{{.Repository}} {{.Size}}' | grep -E 'web-multi|node'   # 64.2 MB tegenover 167 MB
```

---

### Vraag 12 — `/app/dist: no such file or directory`

**Antwoord.** De stage `build` zet `WORKDIR /src`, dus de build levert `/src/dist` op, niet `/app/dist`. De fix: `COPY --from=build /src/dist/<project>/browser /usr/share/nginx/html` (de submap hangt af van de Angular-versie en de projectnaam). Om te diagnosticeren in plaats van te gokken:

`podman build --target build -t dbg .`, en dan `podman run --rm dbg find / -name index.html -path '*dist*'`.

**Waarom.** `COPY --from` kopieert uit het bestandssysteem van de stage, met **absolute** paden binnen die stage. Een verkeerde `WORKDIR` of een onverwachte structuur van `dist/` blijft onzichtbaar tot je erin kijkt.

**Nuance.** Sinds Angular 17 is de standaarduitvoer `dist/<project>/browser/`; daarvoor was het `dist/<project>/`. Met `--target` hoef je niet op je geheugen te vertrouwen.

**Voorbeeld.**

```
podman build --target build -t dbg . && podman run --rm dbg ls -R /src/dist | head
```

---

### Vraag 13 — `RUN mvn test` in de Dockerfile

**Antwoord.** In de build-stage, **na** de compilatie en **vóór** `package` (of als één enkele `mvn package` zonder `-DskipTests`): faalt een test, dan faalt de `RUN`, stopt de build en komt er geen image. Het nadeel: de tests draaien in een geïsoleerde buildcontainer — de CI krijgt geen bruikbaar JUnit-rapport (dat zit in een weggegooide stage, tenzij je het eruit haalt met `--target` of `--output`), een testdatabase is moeilijk bereikbaar (Testcontainers heeft een engine nodig), en de buildtijd van de image omvat voortaan ook de tests, zelfs als je alleen maar wou herbouwen.

**Waarom.** De Dockerfile is een goede poortwachter ("geen image zonder groene tests") maar een slechte rapporteringstool.

**Nuance.** Het gangbare compromis: de CI draait de tests **en** de imagebuild als twee jobs, waarbij de build pas start als de tests slagen; de Dockerfile houdt `-DskipTests` om snel te blijven. Je krijgt het rapport én de garantie, tegen de prijs van een afhankelijkheid van de CI.

**Voorbeeld.**

```
RUN mvn -q test            # rood -> de build stopt hier
RUN mvn -q package -DskipTests
```

---

### Vraag 14 — Eén laag van 250 MB of vijf lagen van 280 MB

**Antwoord.** De image van **280 MB in vijf lagen** rolt sneller uit bij een codewijziging: alleen de vluchtige laag (enkele MB) gaat over het netwerk; de vier andere staan al op de nodes en in de registry. De image van 250 MB in één laag verstuurt bij elke versie opnieuw 250 MB. Het antwoord slaat om zodra de nodes nog **niets** hebben (eerste uitrol, verse node, leeggemaakte registry, of een tagstrategie die telkens alles verandert): dan is 250 < 280 en wint de enkele laag — nipt.

**Waarom.** De overdrachtskost is die van de ontbrekende lagen, niet die van de image. De stabiliteit van de lagen weegt zwaarder dan hun aantal.

**Nuance.** Met `--squash` (Buildah) of een kleinere basis krijg je de 280 MB naar 250 zonder de lagen op te geven: de twee criteria sluiten elkaar niet uit. En de winst bestaat alleen als de stabiele lagen van build tot build **bit voor bit identiek** zijn — dat vraagt reproduceerbare builds (geen niet-vastgepinde `apt-get update` in een "stabiele" laag).

**Voorbeeld.**

```
podman push registry.intern/mijnapp/api:1.5.1     # stabiele blobs: onmiddellijk; alleen de codelaag wordt gekopieerd
```